

Práctica: Servicios REST con Relaciones @OneToMany y @ManyToOne en Oracle

- Implementar un servicio REST en Spring Boot usando Oracle como base de datos.
- Usar relaciones entre tablas con @OneToMany y @ManyToOne.
- Manejar validaciones de datos y mensajes de error personalizados.
- Usar diferentes formas de recibir parámetros (@PathVariable y @RequestParam).

Se tiene un sistema de Biblioteca con dos tablas ya creadas en Oracle:

Tabla: AUTOR

- ID_AUTOR (PK, NUMBER) → Identificador único del autor.
- NOMBRE (VARCHAR2(100)) → Nombre completo del autor.
- NACIONALIDAD (VARCHAR2(50)) → Nacionalidad del autor.
- FECHA_NACIMIENTO (DATE) → Fecha de nacimiento del autor.

Tabla: LIBRO

- ID_LIBRO (PK, NUMBER) → Identificador único del libro.
- TITULO (VARCHAR2(150)) → Título del libro.
- GENERO (VARCHAR2(50)) → Género literario (ej. novela, poesía, cuento).
- ANIO_PUBLICACION (NUMBER(4)) → Año en que se publicó el libro.
- PRECIO (NUMBER(10,2)) → Precio de venta del libro.
- ID_AUTOR (FK, NUMBER) → Llave foránea que referencia a AUTOR(ID_AUTOR).

Tu tarea es implementar los servicios REST para manipular estas entidades y su relación.

Instrucciones

1. Configuración del proyecto

Crea un proyecto en Spring Boot con las dependencias necesarias:

- Spring Web
- Spring Data JPA
- Validation
- Oracle Driver

Configura el archivo `application.properties` para conectarte a tu base de datos Oracle.

(Usuario, contraseña y URL ya los conoces de prácticas anteriores).

2. Creación de entidades

Crea una clase `Autor` que represente la tabla `AUTOR`.

- Incluye el campo `id` como llave primaria.
- Incluye el campo `nombre`.
- Aplica una validación que obligue a que el nombre no esté vacío.

- Define una relación @OneToMany hacia la entidad Libro.

Crea una clase Libro que represente la tabla LIBRO.

- Incluye el campo id como llave primaria.
- Incluye el campo titulo.
- Aplica una validación que obligue a que el título no esté vacío.
- Define una relación @ManyToOne con la entidad Autor.

3. Repositorios

- Crea un repositorio para Autor que extienda JpaRepository.
- Crea un repositorio para Libro que extienda JpaRepository.

4. Service

- Crear la interfaz con los métodos a implementar
- Inyecta el repositorio e implementa la interfaz del service

5. Endpoints a implementar

Implementa los siguientes endpoints en un controlador REST llamado BibliotecaController:

1. Crear Autor

Método: POST /api/autores

- Recibe en el cuerpo el nombre del autor.
- Devuelve un mensaje de confirmación y los datos del autor creado.

2. Crear Libro para un Autor (usando PathVariable)

Método: POST /api/autores/{autorId}/libros

- Recibe en el cuerpo el título del libro.
- El ID del autor se recibe como PathVariable.
- Devuelve un mensaje de confirmación y los datos del libro creado.

3. Buscar un Autor por ID (usando RequestParam)

Método: GET /api/autores?id=XX

- Recibe el ID como RequestParam.

- Devuelve un mensaje de confirmación y los datos del autor.

4. Obtener todos los Libros de un Autor (usando PathVariable)

Método: GET /api/autores/{autorId}/libros

- El ID del autor se recibe como PathVariable.
- Devuelve todos los libros asociados a ese autor.

5. (Opcional Extra) Obtener el Autor de un Libro

Método: GET /api/libros/{libroId}/autor

- El ID del libro se recibe como PathVariable.
- Devuelve la información del autor de ese libro.

6. Pruebas en Postman

- Realiza las siguientes pruebas:
- Crear un autor con un nombre válido.
- Crear un autor con un nombre vacío (debe dar error de validación).
- Crear un libro para un autor existente.

- Crear un libro para un autor que no exista (debe dar error).
- Buscar un autor con RequestParam.
- Obtener los libros de un autor existente.
- Obtener los libros de un autor inexistente (debe dar error).
- (Opcional) Obtener el autor de un libro.